

**Machine Learning
with
Python Programming
A
Industrial Training Report
submitted
in partial fulfilment
for the award of the degree of
Bachelor of Technology
in department of Computer Science
with specialization in Computer Science**



Submitted To:

Mr. Atul Gaur

Astt. Prof, CSE

Submitted By:

Amit Sain

Roll no:22ELDCS300

LIET-Alwar

Department of Computer Science & Engineering

Laxmi Devi Institute of Engineering & Technology, Alwar

Bikaner Technical University [2022-23]

Acknowledgement

I would like to express my heartfelt gratitude to all those who supported and guided me throughout my online training in **Machine Learning with Python Programming**. This enriching journey has been made possible with the valuable contributions of many individuals and organizations.

First and foremost, I am profoundly thankful to my trainers and mentors for their expert guidance, insightful lectures, and patience in clarifying my doubts. Their dedication to teaching and their deep knowledge of machine learning concepts have been instrumental in shaping my understanding of the subject.

I am also grateful to the online training platform and its team for providing a well-structured curriculum, interactive learning modules, and hands-on projects. The access to comprehensive resources and tools has enhanced my practical skills significantly.

This training has been a pivotal step in my educational and professional journey, and I am truly grateful for the opportunity to explore the fascinating world of machine learning using Python programming.

Amit Sain

22ELDCS300

Candidate's Declaration

I hereby declare that the Industrial/Field Project, which is being intern in the field, entitled “**Python Programming & Machine Learning**” in partial fulfilment for the award of Degree of “Bachelor of Technology” in Department of Computer Science & Engineering with Specialization in Computer Science Engineering , and submitted to the **Department of Computer Science & Engineering, Laxmi Devi Institute of Engineering & Technology, Alwar**, Bikaner Technical University is a record of my own Learning and internship carried under the Guidance of **Mr. Atul Gaur, Project Report co-ordinator(CSE)** : Laxmi Devi institute of Engineering & Technology, Alwar.

I have not submitted the matter presented in this field work report anywhere for the award of any other Degree.

Student Name: AMIT SAIN

Branch: Computer Science

Roll no: 22ELDCS300

Laxmi Devi Institute of Engineering & Technology, Alwar

Counter Signed by

Guide Name: Mr. Atul Gaur

Department: Computer Science & Engineering Department

Laxmi Devi Institute of Engineering & Technology, Alwar

CERTIFICATE OF COMPLETION

Amit Sain

has successfully completed the online course:

Machine Learning

This professional has demonstrated initiative and a commitment to deepening their skills and advancing their career. Well done!

5th December 2024



Krishna Kumar
CEO, Simplilearn



Abstract

This report investigates the impact of Python language & Machine Learning on IT sector. This report explores the synergy between Python and ML, focusing on Python's key features, including its versatility, ease of learning, and integration capabilities.

Key findings reveal that essential Python libraries such as TensorFlow, Scikit-learn, and Pandas, NumPy, Matplotlib, which enable efficient data manipulation, model development, and deployment, providing insights into real-world applications where Python-driven ML solutions have transformed industries, including healthcare, finance, and automation.

The report concludes with recommendations for outlining future opportunities for Python in the evolving ML landscape. This study underscores Python's pivotal role in democratizing ML and advancing artificial intelligence technologies.

Furthermore, it discusses current challenges, such as computational limitations, security and privacy concerns while data bias and fairness while outlining future opportunities for Python in the evolving ML landscape.

Table of contents

1. INTRODUCTION	8
1.1 What is Python?	8
1.2 History of Python	9
1.3 Why Python called Python?	9
1.4 Key Features of Python	9
1.5 Installing Python	12
1.6 Object Oriented Programming.....	14
2. PYTHON DATA TYPES	20
2.1 Variables in Python	20
2.2 Strings in Python	20
2.3 Lists in Python	21
2.4 Tuples in Python.....	21
2.5 Sets in Python	21
2.6 Dictionary in Python.....	22
2.7 Boolean in Python.....	22
3. CONTROL FLOW IN PYTHON	24
3.1 Conditional Statements in Python	24
3.2 Looping in Python	27
4. FUNCTIONS IN PYTHON	29
4.1 Function	29
4.2 Lambda Function.....	31
5. MACHINE LEARNING	32
5.1 Introduction to Machine Learning	32
5.2 Types of Machine Learning	33
5.3 Framework and Libraries in ML.....	35
5.4 Challenges to ML.....	39
6. FUTURE TRENDS.....	42
6.1 Scope of ML.....	42
6.2 Benefits of ML	43
CONCLUSION.....	46
REFERENCES.....	47

Table of Figures

Figure 1. Python	10
Figure 2. Download Python.....	13
Figure 3. Install Python	14
Figure 4. Main Concepts of OOPS	15
Figure 5. Classes and Objects	16
Figure 6. Methods	18
Figure 7. Strings.....	20
Figure 8. Lists.....	21
Figure 9. Tuples	21
Figure 10. Dictionary	22
Figure 11. Boolean Expressions	23
Figure 12. Conditional Statements.....	27
Figure 13. Functions.....	29
Figure 14. Arguments in Python	30
Figure 15. Supervised Learning	33
Figure 16. Unsupervised Learning.....	34
Figure 17. TensorFlow	36
Figure 18. NumPy.....	37
Figure 19. Pandas	38
Figure 20. Matplotlib.....	39
Figure 21. Scopes of ML	43
Figure 22. Benefits of ML	45

1. INTRODUCTION

1.1 What is Python?

Python is a high-level, general-purpose and a very popular programming language. Python programming language (latest Python 3) is being used in web development, Machine Learning applications, along with all cutting-edge technology in Software Industry. Python Programming Language is very well suited for Beginners, also for experienced programmers with other programming languages like C++ and Java.

This specially designed Python tutorial will help you learn Python Programming Language in most efficient way, with the topics from basics to advanced (like Web-scraping, Django, Deep-Learning, etc.) with examples.

Below are some facts about Python Programming Language:

1. Python is currently the most widely used multi-purpose, high-level programming language.
2. Python allows programming in Object-Oriented and Procedural paradigms.
3. Python programs generally are smaller than other programming languages like Java. Programmers would have to type relatively less and indentation requirement of the language, makes them readable all the time.
4. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.
5. The biggest strength of Python is huge collection of standard libraries which can be used for the following:
 - Machine Learning
 - GUI Applications (like, Tkinter, PyQt etc.)
 - Web frameworks like Django (used by YouTube, Instagram, Dropbox)
 - Image processing (like OpenCv, Pillow)
 - Web scraping (like Scrapy, BeautifulSoup, Selenium)
 - Test frameworks
 - Multimedia
 - Scientific computing
 - Text processing and many more

1.2 History of Python

In the late 1980s, history was about to be written. It was that time when working on Python started. Soon after that, Guido Van Rossum began doing its application-based work in December of 1989 at Centrum Wiskunde & Informatica (CWI) which is situated in the Netherlands. It was started as a hobby project because he was looking for an interesting project to keep him occupied during Christmas.

The programming language in which Python is said to have succeeded is ABC Programming Language, which had interfacing with the Amoeba Operating System and had the feature of exception handling. He had already helped create ABC earlier in his career and had seen some issues with ABC but liked most of the features. After that what he did was very clever. He had taken the syntax of ABC, and some of its good features. It came with a lot of complaints too, so he fixed those issues completely and created a good scripting language that had removed all the flaws.

1.3 Why Python called Python?

The inspiration for the name came from the BBC's TV Show – ‘Monty Python's Flying Circus’, as he was a big fan of the TV show and also he wanted a short, unique and slightly mysterious name for his invention and hence he named it Python!

1.4 Key Features of Python

1. GUI Programming Support

Python supports GUI programming through libraries such as Tkinter, PyQt, and Kivy, making it easy to create user-friendly desktop applications. These libraries provide tools and widgets to build interactive, cross-platform graphical user interfaces.

2. Object-Oriented Programming

Python is fully object-oriented, enabling developers to create classes and objects to model real-world entities. Features like inheritance, encapsulation, and polymorphism are supported, making it easier to build complex applications using reusable and organized code.

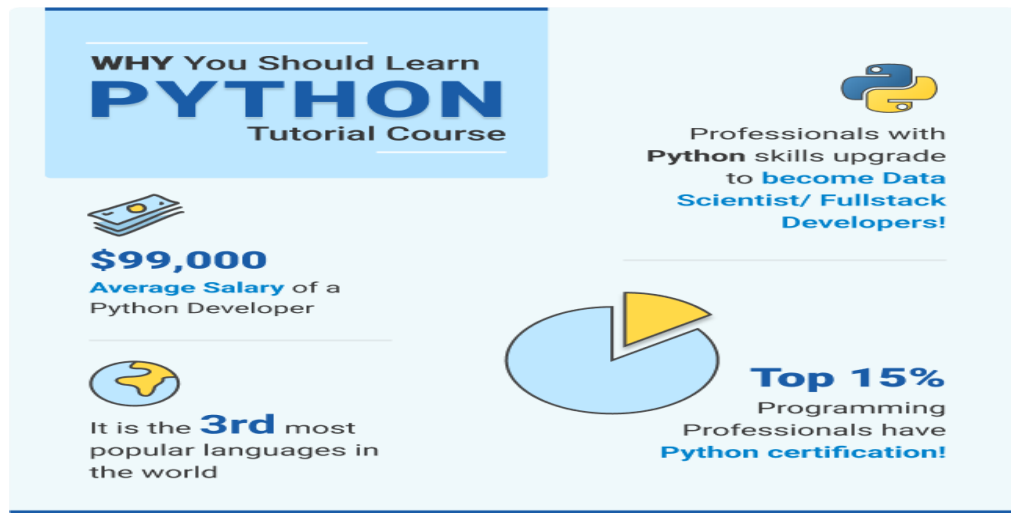


Figure 1. Python

3. Dynamic Language

Python is dynamically typed, meaning that variable types are determined at runtime. This allows for more flexibility in coding since developers do not need to declare variable types explicitly, speeding up the development process.

4. Easy to Code

Python's simple and readable syntax makes it one of the most beginner-friendly programming languages. It emphasizes readability and reduces code complexity, enabling quick development and accessible learning for newcomers.

5. Free and Open-Source

Python is free to download, use, and distribute, as it is open-source under the Python Software Foundation License. This makes it accessible to everyone, fostering innovation and collaborative improvements within the community.

6. High-level Programming Language

As a high-level programming language, Python abstracts complex tasks, allowing developers to focus more on problem-solving rather than low-level details like memory management or hardware architecture.

7. Interpreted Language

Python is an interpreted language, meaning code is executed line by line at runtime without requiring prior compilation. This simplifies debugging and testing, enhancing the development workflow.

8. Standard Library

This is one of the key features of Python. Python has a vast standard library that provides built-in modules and functions for tasks such as file I/O, web development, data manipulation, regular expressions, and more. This reduces the need for external dependencies and speeds up development.

9. Community Support

Python boasts a large and active community of developers. This means extensive documentation, tutorials, forums, and open-source libraries are readily available, making it easier for developers to find support and contribute to the language.

10. Portable Language

Python code can run on various operating systems like Windows, Linux, macOS, and more without modification. This cross-platform portability enables developers to create software that can be executed in multiple environments seamlessly.

11. Databases

Python offers excellent support for database connectivity. Libraries like SQLite3, SQLAlchemy, and PyMySQL allow developers to interact with databases, execute queries, and manage data efficiently, making Python suitable for building database-driven applications.

12. Extensible

Python can be extended with code written in languages like C or C++. This allows for enhanced performance and the ability to use existing libraries and modules from other ecosystems within Python programs.

13. Cross Platform

Python is cross-platform, allowing the same codebase to be run on different operating systems without modification. This is essential for developing software that needs to be compatible across various devices and platforms.

14. Enhancements to Python Types

Python supports enhancements and type hints through tools like typing and type checkers such as mypy. This allows developers to specify types for variables, function parameters, and return values, leading to better readability and fewer runtime errors.

15. Frontend and Backend Development

While primarily used for backend development in frameworks like Django and Flask, Python can also be used in the front end through frameworks like Brython or Pyodide. This flexibility makes Python a versatile choice for full-stack development.

1.5 Installing Python

Here we will be discussing how to get the answer to all questions related to installing Python on Windows/Linux/macOS. Python was developed by Guido van Rossum in the early 1990s, and its latest version is 3.10.8, we can simply call it Python3. To understand how to install Python You need to know *What Python is* and where it is actually installed in your system. Let's consider a few points:

- Python is a widely used general-purpose, high-level programming language.
- Every Release of Python is open source. Python releases have also been GPL-compatible.
- Any version of Python can be downloaded from Python Software Foundation website at python.org.
- Most the languages, notably Linux provides a package manager through which you can directly install Python on your Operating System

In this Python tutorial on Installation and Setup, you'll see how to install Python on Windows, macOS, Linux, iOS, and Android.

How to install Python on Windows?

Since windows don't come with Python preinstalled, it needs to be installed explicitly. Here we will define step by step tutorial on How to install Python on Windows. Follow the steps below:

Download Python Latest Version from python.org

Step 1: First and foremost, step is to open a browser and type

<https://www.python.org/downloads/>

Step 2: Click on Download Python 3.13.0 (the latest stable release as of now is Python 3.13.0).



Figure 2. Download Python

Step 3: On this page move to Files and click on Windows x86-64 executable installer for 64-bit or Windows x86 executable installer for 32-bit.

Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b00c8cf6d9ec0b9abe83184a40729a2	7504391	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76debbdb3043a583e563400	26680368	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28cb1c608bbd73ae8e53a3bd351b4bd2	1362904	SIG
Windows x86 embeddable zip file	Windows		9fab3b81f8841879fda94133574139d8	6741626	SIG
Windows x86 executable installer	Windows		33cc602942a54446a3d6451476394789	25663848	SIG
Windows x86 web-based installer	Windows		1b670cfa5d317df82c30983ea371d87c	1324608	SIG

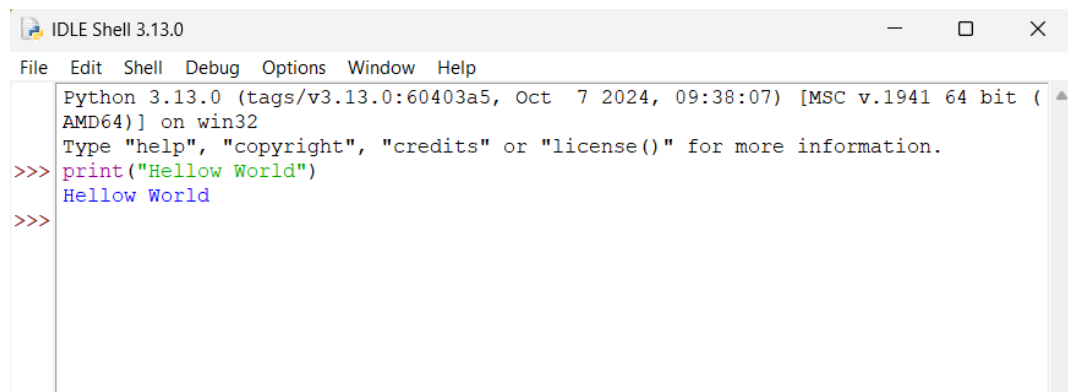
Here we are providing the installation process of Python 3.13.0 on Windows

- Run the Python Installer for how to install python on windows downloads folder
- Make sure to mark Add Python.exe to PATH otherwise you will have to do it explicitly. It will start installing python on windows.



Figure 3. Install Python

After installation is complete click on Close. Bingo!! Python is installed. Now go to windows and type IDLE.



- This is Python Interpreter also called Python Shell. I printed “Hello World”, python is working smoothly.
- The three greater than >>> sign is called Python command prompt, where we write our program and with a single enter key, it will give result so instantly.

1.6 Object Oriented Programming

In Python, object-oriented Programming (OOPs) is a programming paradigm that uses objects and classes in programming. It aims to implement real-world entities like inheritance, polymorphisms, encapsulation, etc. in the

programming. The main concept of OOPs is to bind the data and the functions that work on that together as a single unit so that no other part of the code can access this data.

Main Concepts of Object-Oriented Programming (OOPS):

- Class
- Objects
- Polymorphism
- Encapsulation
- Inheritance
- Data Abstraction

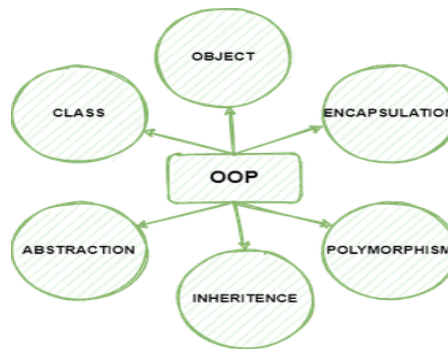


Figure 4. Main Concepts of OOPS

1. Class

A class is a collection of objects. A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and methods.

To understand the need for creating a class let's consider an example, let's say you wanted to track the number of dogs that may have different attributes like breed, age. If a list is used, the first element could be the dog's breed while the second element could represent its age. Let's suppose there are 100 different dogs, then how would you know which element is supposed to be which? What if you wanted to add other properties to these dogs? This lacks organization and it is the exact need for classes.

Example 1: Creating a class and object with class and instance attributes

```

class Dog:

    # class attribute
    attr1 = "mammal"

    # Instance attribute
    def __init__(self, name):
        self.name = name

# Driver code
# Object instantiation
Rodger = Dog("Rodger")
Tommy = Dog("Tommy")

# Accessing class attributes
print("Rodger is a {}".format(Rodger.__class__.attr1))
print("Tommy is also a {}".format(Tommy.__class__.attr1))

# Accessing instance attributes
print("My name is {}".format(Rodger.name))
print("My name is {}".format(Tommy.name))

```

Figure 5. Classes and Objects

Output

```

Rodger is a mammal
Tommy is also a mammal
My name is Rodger
My name is Tommy

```

2.Objects

The object is an entity that has a state and behaviour associated with it. It may be any real-world object like a mouse, keyboard, chair, table, pen, etc. Integers, strings, floating-point numbers, even arrays, and dictionaries, are all objects. More specifically, any single integer or any single string is an object. The number 12 is an object, the string “Hello, world” is an object, a list is an

object that can hold other objects, and so on. You've been using objects all along and may not even realize it.

An object consists of:

- **State:** It is represented by the attributes of an object. It also reflects the properties of an object.
- **Behaviour:** It is represented by the methods of an object. It also reflects the response of an object to other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

3.Inheritance

Inheritance is the capability of one class to derive or inherit the properties from another class. The class that derives properties is called the derived class or child class and the class from which the properties are being derived is called the base class or parent class. The benefits of inheritance are:

- It represents real-world relationships well.
- It provides the reusability of a code. We don't have to write the same code again and again. Also, it allows us to add more features to a class without modifying it.
- It is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.

Types of Inheritance –

- **Single Inheritance:**

Single-level inheritance enables a derived class to inherit characteristics from a single-parent class.

- **Multilevel Inheritance:**

Multi-level inheritance enables a derived class to inherit properties from an immediate parent class which in turn inherits properties from his parent class.

- **Hierarchical Inheritance:**

Hierarchical level inheritance enables more than one derived class to inherit properties from a parent class.

- **Multiple Inheritance:**

Multiple level inheritance enables one derived class to inherit properties from more than one base class.

4.Polymorphism

Polymorphism simply means having many forms. For example, we need to determine if the given species of birds fly or not, using polymorphism we can do this using a single function.

5.Encapsulation

Encapsulation is one of the fundamental concepts in object-oriented programming (OOP). It describes the idea of wrapping data and the methods that work on data within one unit. This puts restrictions on accessing variables and methods directly and can prevent the accidental modification of data. To prevent accidental change, an object's variable can only be changed by an object's method. Those types of variables are known as private variables.

A class is an example of encapsulation as it encapsulates all the data that is member functions, variables, etc.

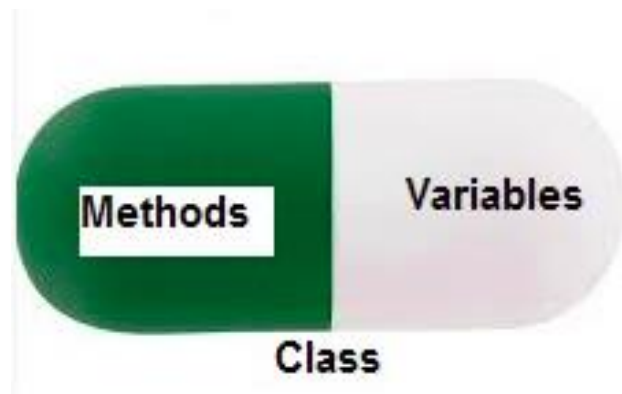


Figure 6. Methods

6.Data Abstraction

It hides the unnecessary code details from the user. Also, when we do not want to give out sensitive parts of our code implementation and this is where data abstraction came.

Data Abstraction in Python can be achieved through creating abstract classes.

2. PYTHON DATA TYPES

2.1 Variables in Python

In many programming languages, variables are statically typed. That means a variable is initially declared to have a specific data type, and any value assigned to it during its lifetime must always have that type.

Variables in Python are not subject to this restriction. In Python, a variable may be assigned a value of one type and then later re-assigned a value of a different type:

```
>>> var = 23.5
```

```
>>> print(var)
```

```
23.5
```

```
>>> var = "Now I'm a string"
```

```
>>> print(var)
```

```
Now I'm a string
```

2.2 Strings in Python

A string is a sequence of characters that can be a combination of letters, numbers, and special characters. It can be declared in python by using single quotes, double quotes, or even triple quotes. These quotes are not a part of a string, they define only starting and ending of the string. Strings are immutable, i.e., they cannot be changed. Each element of the string can be accessed using indexing or slicing operations.

```
# Single quotes
string1 = 'Hello, World!'

# Double quotes
string2 = "Hello, World!"

# Triple quotes for multiline strings
string3 = '''Hello,
World!'''
```

Figure 7. Strings

2.3 Lists in Python

Lists are one of the most powerful data structures in python. Lists are sequenced data types. In Python, an empty list is created using list () function. They are just like the arrays declared in other languages. But the most powerful thing is that list need not be always homogeneous. A single list can contain strings, integers, as well as other objects. Lists can also be used for implementing stacks and queues. Lists are mutable, i.e., they can be altered once declared. The elements of list can be accessed using indexing and slicing operations.

```
# Creating a list of fruits
fruits = ["apple", "banana", "cherry"]
print(fruits)

# Accessing elements in a list
print(fruits[0]) # Output: apple
print(fruits[2]) # Output: cherry
```

Figure 8. Lists

2.4 Tuples in Python

A tuple is a sequence of immutable Python objects. Tuples are just like lists with the exception that tuples cannot be changed once declared. Tuples are usually faster than lists.

Creating Tuples

Tuples are created by placing items inside parentheses `()`, separated by commas. For example:

```
mytuple = ("apple", "banana", "cherry")
print(mytuple)
```

You can also create a tuple using the `tuple()` constructor:

```
thistuple = tuple(("apple", "banana", "cherry"))
print(thistuple)
```

To create a tuple with a single item, you must include a trailing comma:

```
single_item_tuple = ("apple",)
print(type(single_item_tuple)) # Output: <class 'tuple'>
```

Figure 9. Tuples

2.5 Sets in Python

In Python, Set is an unordered collection of data type that is iterable, mutable and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

2.6 Dictionary in Python

In python, the dictionary is like hash or maps in other languages. It consists of key-value pairs. The value can be accessed by a unique key in the dictionary. (Before python 3.7 dictionary used to be unordered meant key-value pairs are not stored in the order they are inputted into the dictionary but from python 3.7 they are stored in order like the first element will be stored in the first position and the last at the last position.)

Creating a Dictionary

You can create a dictionary using curly braces `{}` with key-value pairs separated by commas. For example:

```
# Creating a dictionary
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print(thisdict)
```

Alternatively, you can use the `dict()` constructor:

```
# Using the dict() constructor
thisdict = dict(name="John", age=36, country="Norway")
print(thisdict)
```

Figure 10. Dictionary

2.7 Boolean in Python

Data type with one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true), and those equal to False are falsie (false). But non-Boolean objects can be evaluated in Boolean context as well and determined to be true or false. It is denoted by the class bool.

Boolean Values and Expressions

Booleans are often used in comparisons and logical operations. For example:

```
print(10 > 9) # True
print(10 == 9) # False
print(10 < 9) # False
```

When you run a condition in an `if` statement, Python returns **True** or **False**:

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

Evaluating Values and Variables

The `bool()` function allows you to evaluate any value and returns **True** or **False**:

```
print(bool("Hello")) # True
print(bool(15)) # True
```

Figure 11. Boolean Expressions

3. CONTROL FLOW IN PYTHON

3.1 Conditional Statements in Python

They're come situations in real life when we need to make some decisions and based on these decisions, we decide what should we do next. Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code. Decision-making statements in programming languages decide the direction of the flow of program execution.

- **if statement:**

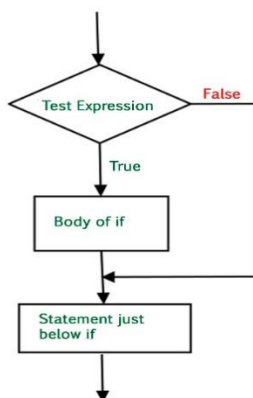
if statement is the simplest decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e. if a certain condition is true then a block of statement is executed otherwise not.

if condition:

Statements to execute if

condition is true

Here, the condition after evaluation will be either true or false. if the statement accepts Boolean values – if the value is true then it will execute the block of statements below it otherwise not. We can use *condition* with bracket ‘(“)’ also.



- **if-else:**

The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else if the condition is false. Here comes the *else* statement. We can use the *else* statement with *if* statement to execute a block of code when the condition is false.

Syntax:

if (condition):

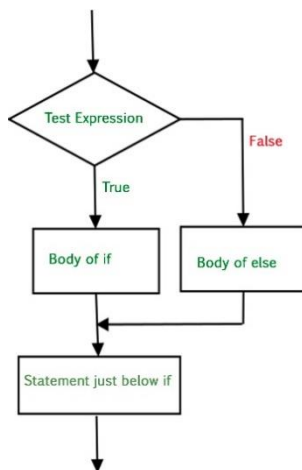
Executes this block if

condition is true

else:

Executes this block if

condition is false



- **nested-if:**

A nested if is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, Python allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.

Syntax:

if (condition1):

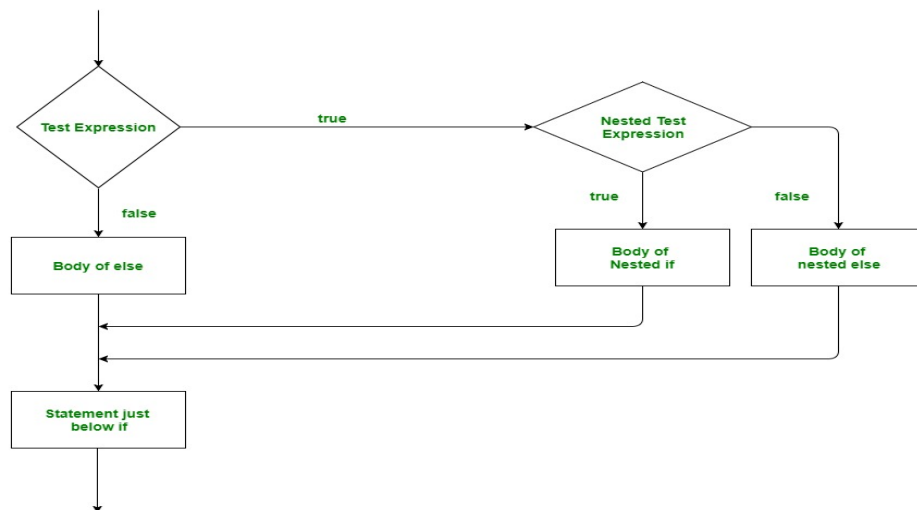
Executes when condition1 is true

if (condition2):

Executes when condition2 is true

if Block is end here

if Block is end here



- **if-elif-else ladder:**

Here, a user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

Syntax:

if (condition):

statement

elif (condition):

statement

•

•

else:

statement

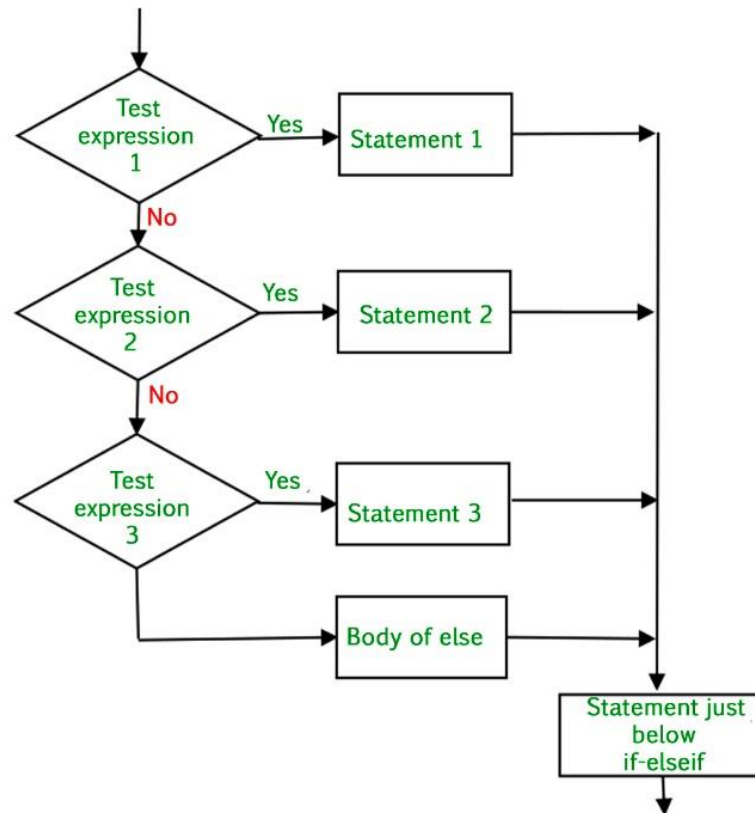


Figure 12. Conditional Statements

3.2 Looping in Python

- **While loop**

In Python, a while loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.

Python while loop syntax

```
while expression:
    statement(s)
```

All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

- **For loop**

For loop are used for sequential traversal. For example: traversing a list or string or array etc. In Python, there is “for in” loop which is similar

to foreach loop in other languages. Let us learn how to use for loops in Python for sequential traversals with examples.

For Loop Syntax:

for iterator_var in sequence

statements(s):

It can be used to iterate over a range and iterators.

- **Nested loop**

Python programming language allows to use one loop inside another loop which is called nested loop. Following section shows few examples to illustrate the concept.

Nested Loops Syntax:

```
for iterator_var in sequence:  
    for iterator_var in sequence:  
        statements(s)  
    statements(s)
```

The syntax for a nested while loop statement in the Python programming language is as follows:

```
while expression:  
    while expression:  
        statement(s)  
    statement(s)
```

A final note on loop nesting is that we can put any type of loop inside of any other type of loops in Python. For example, a for loop can be inside a while loop or vice versa.

4. FUNCTIONS IN PYTHON

4.1 Function

Python Functions is a block of statements that return the specific task.

The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it repeatedly.

Syntax: Python Functions

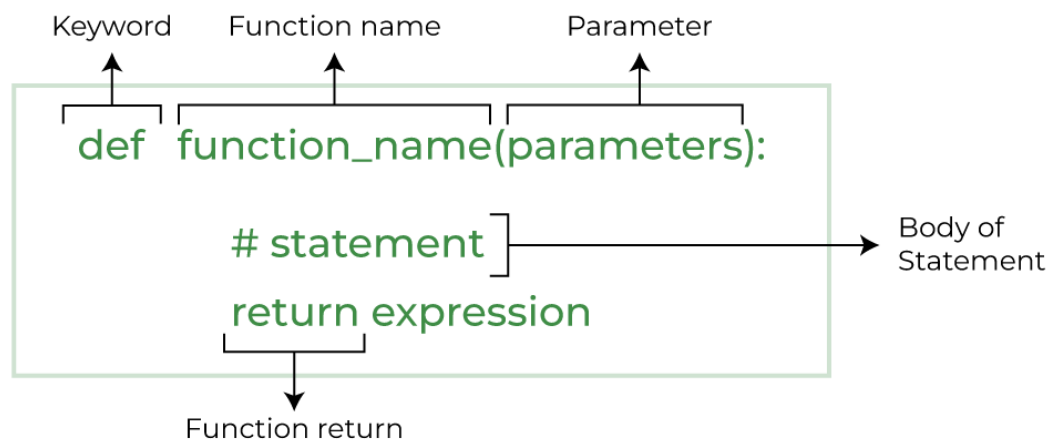


Figure 13. Functions

Creating a Python Function:

We can create a Python function using the `def` keyword.

```
# A simple Python function  
  
def fun():  
    print("Welcome to GFG")
```

Calling a Python Function:

After creating a function, we can call it by using the name of the function followed by parenthesis containing parameters of that function.

```
# A simple Python function
def fun():
    print("Welcome to GFG")

# Driver code to call a function
fun()
```

Defining and calling a function with parameters:

If you have experience in C/C++ or Java, then you must be thinking about the *return type* of the function and *data type* of arguments. That is possible in Python as well (specifically for Python 3.5 and above).

Syntax: Python Function with parameters

```
def function_name(parameter: data_type) -> return_type:
    """Doctring"""
    # body of the function
    return expression
```

Arguments of a Python Function:

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma.

In this example, we will create a simple function to check whether the number passed as an argument to the function is even or odd.

```
# A simple Python function to check
# whether x is even or odd

def evenOdd(x):
    if (x % 2 == 0):
        print("even")
    else:
        print("odd")

# Driver code to call the function
evenOdd(2)
evenOdd(3)
```

Figure 14. Arguments in Python

4.2 Lambda Function

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.

Syntax

lambda arguments: expression

The expression is executed and the result is returned:

5. MACHINE LEARNING

5.1 Introduction to Machine Learning

Machine learning (ML) is a type of artificial intelligence (AI) that allows computers to learn without being explicitly programmed. It involves feeding data into algorithms that can then identify patterns and make predictions on new data. Machine learning is used in a wide variety of applications, including image and speech recognition, natural language processing, and recommender systems.

Examples of Machine Learning in Action:

Machine learning is woven into the fabric of our daily lives. Here are some examples to illustrate its diverse applications

- **Filtering Your Inbox:** Spam filters use machine learning to analyse emails and identify spam based on past patterns. They learn from emails you mark as spam and not spam, becoming more accurate over time.
- **Recommending Your Next Purchase:** E-commerce platforms and streaming services use machine learning to analyse your purchase history and viewing habits. This allows them to recommend products and shows you're more likely to enjoy.
- **Smart Reply in Emails:** Machine learning powers features like "Smart Reply" in Gmail, suggesting short responses based on the content of the email.
- **Grouping Customers:** Machine learning can analyse customer data (purchase history, demographics) to identify customer segments with similar characteristics. This helps businesses tailor marketing campaigns and product offerings.
- **Anomaly Detection:** Financial institutions use machine learning to detect unusual spending patterns on your credit card, potentially indicating fraudulent activity.
- **Image Classification in Photos:** Facial recognition in photos on social media platforms is powered by machine learning algorithms trained on vast amounts of labelled data.
- **Self-Driving Cars:** These rely on reinforcement learning, a type of machine learning where algorithms learn through trial and error in a simulated environment.
- **Medical Diagnosis:** Machine learning algorithms can analyse medical images (X-rays, MRIs) to identify abnormalities and aid doctors in diagnosis.

5.2 Types of Machine Learning

- **Supervised Machine Learning**

Supervised learning is defined as when a model gets trained on a “Labelled Dataset”. Labelled datasets have both input and output parameters. In Supervised Learning algorithms learn to map points between inputs and correct outputs. It has both training and validation datasets labelled.

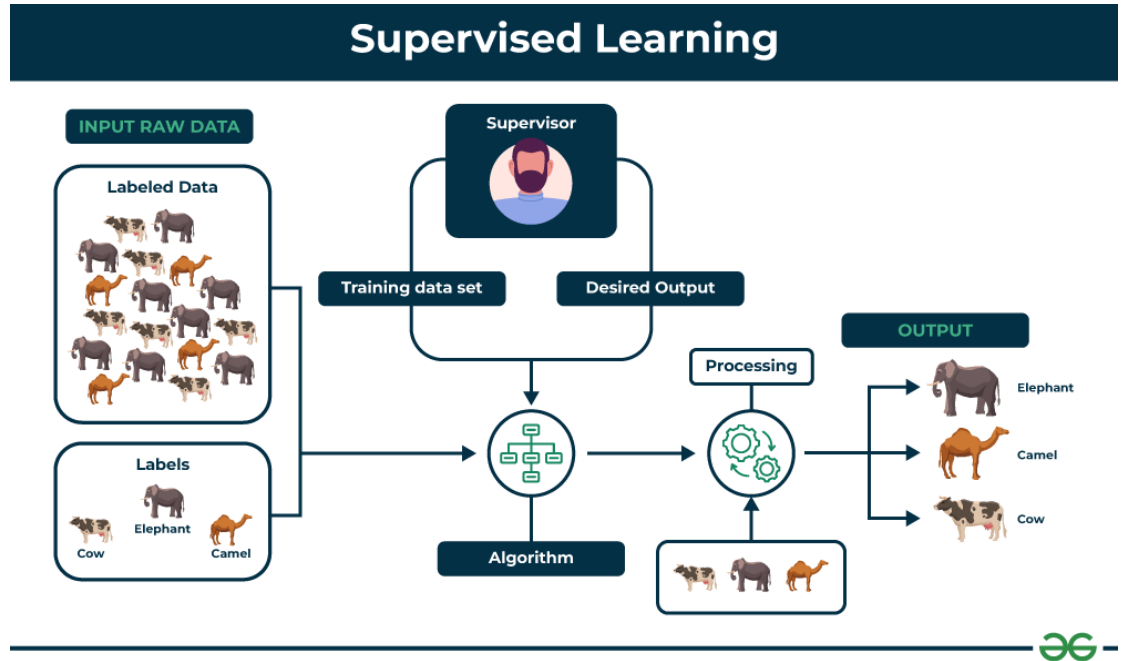


Figure 15. Supervised Learning

- **Unsupervised Machine Learning**

Unsupervised Learning Unsupervised learning is a type of machine learning technique in which an algorithm discovers patterns and relationships using unlabelled data. Unlike supervised learning, unsupervised learning doesn't involve providing the algorithm with labelled target outputs. The primary goal of Unsupervised learning is often to discover hidden patterns, similarities, or clusters within the data, which can then be used for various purposes, such as data exploration, visualization, dimensionality reduction, and more.

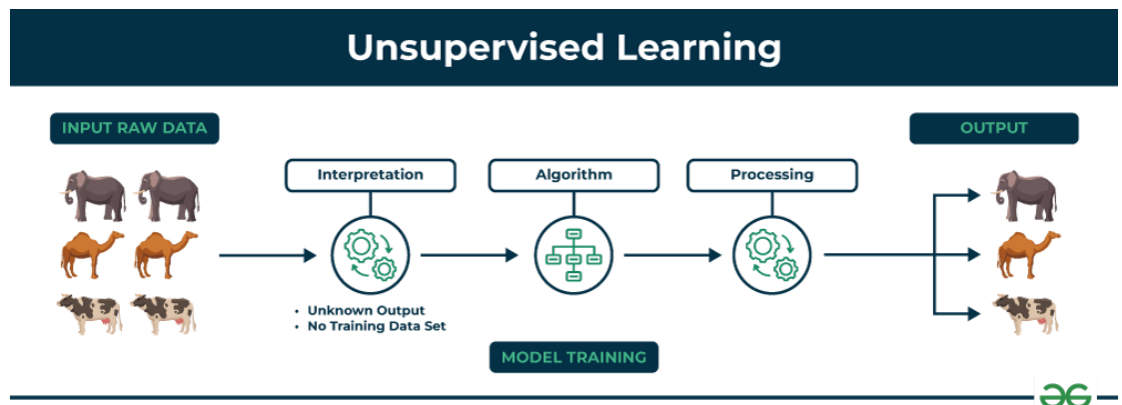
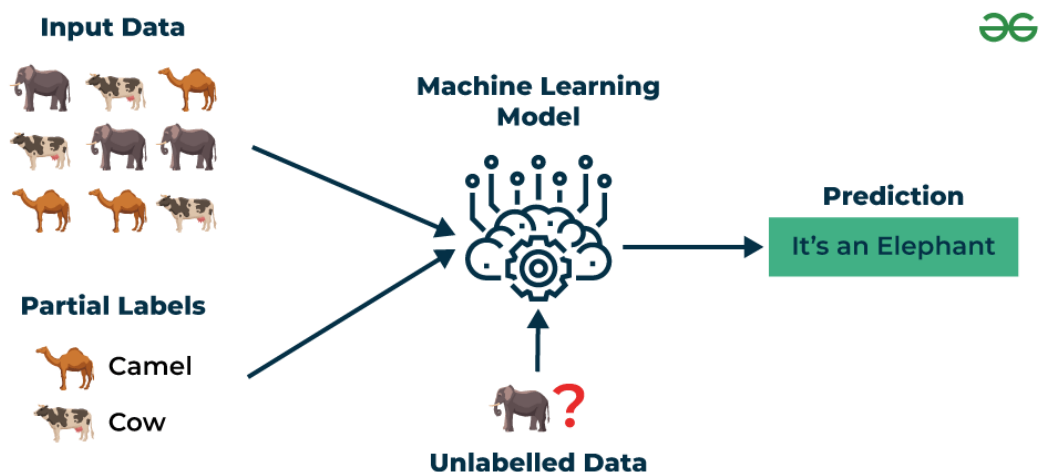


Figure 16. Unsupervised Learning

- **Semi-Supervised Learning**

Semi-Supervised learning is a machine learning algorithm that works between the supervised and unsupervised learning so it uses both **labelled and unlabelled** data. It's particularly useful when obtaining labelled data is costly, time-consuming, or resource-intensive. This approach is useful when the dataset is expensive and time-consuming. Semi-supervised learning is chosen when labelled data requires skills and relevant resources in order to train or learn from it.

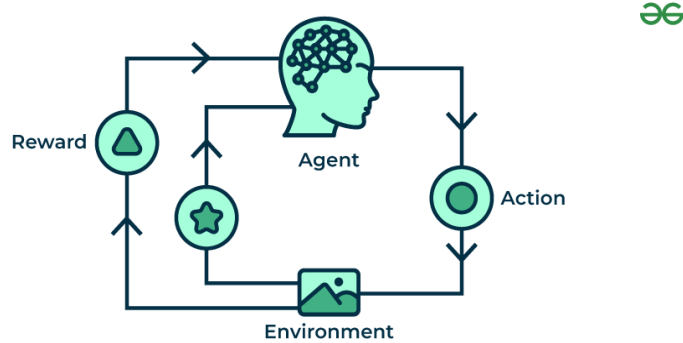
We use these techniques when we are dealing with data that is a little bit labelled and the rest large portion of it is unlabelled. We can use the unsupervised techniques to predict labels and then feed these labels to supervised techniques. This technique is mostly applicable in the case of image data sets where usually all images are not labelled.



- **Reinforcement Machine Learning**

Reinforcement machine learning algorithm is a learning method that interacts with the environment by producing actions and discovering errors. **Trial, error, and delay** are the most relevant characteristics of

reinforcement learning. In this technique, the model keeps on increasing its performance using Reward Feedback to learn the behaviour or pattern. These algorithms are specific to a particular problem e.g. Google Self Driving car, AlphaGo where a bot competes with humans and even itself to get better and better performers in Go Game. Each time we feed in data, they learn and add the data to their knowledge which is training data. So, the more it learns the better it gets trained and hence experienced.



5.3 Framework and Libraries in ML

1) TensorFlow

TensorFlow is a very popular open-source library for high performance numerical computation developed by the Google Brain team in Google. As the name suggests, TensorFlow is a framework that involves defining and running computations involving tensors. It can train and run deep neural networks that can be used to develop several AI applications. TensorFlow is widely used in the field of deep learning research and application.

Python

```
1 # Python program using TensorFlow
2 # for multiplying two arrays
3
4 # import `tensorflow`
5 import tensorflow as tf
6
7 # Initialize two constants
8 x1 = tf.constant([1, 2, 3, 4])
9 x2 = tf.constant([5, 6, 7, 8])
10
11 # Multiply
12 result = tf.multiply(x1, x2)
13
14 # Initialize the Session
15 sess = tf.Session()
16
17 # Print the result
18 print(sess.run(result))
19
20 # Close the session
21 sess.close()
```

Output:

```
[ 5 12 21 32]
```

Figure 17. TensorFlow

Features:

- End-to-end platform with support for deployment across cloud, mobile, and browsers.
- High-level APIs (e.g., Keras) for simplified model building.
- Strong support for distributed training and scalability.

Drawbacks:

- Steep learning curve, especially for beginners.
- Debugging can be complex.
- Can be resource-intensive for small-scale projects.

2) Scikit-learn

Scikit-learn is one of the most popular ML libraries for classical ML Algorithms. It is built on top of two basic Python libraries, viz., NumPy and

SciPy. Scikit-learn supports most of the supervised and unsupervised learning algorithms. Scikit-learn can also be used for data mining and data-analysis, which makes it a great tool who is starting out with ML.

Example: Decision Tree Classifier

Features:

- Simple and user-friendly for classical machine learning algorithms.
- Wide variety of models for classification, regression, and clustering.
- Seamless integration with other Python libraries like NumPy and Pandas.

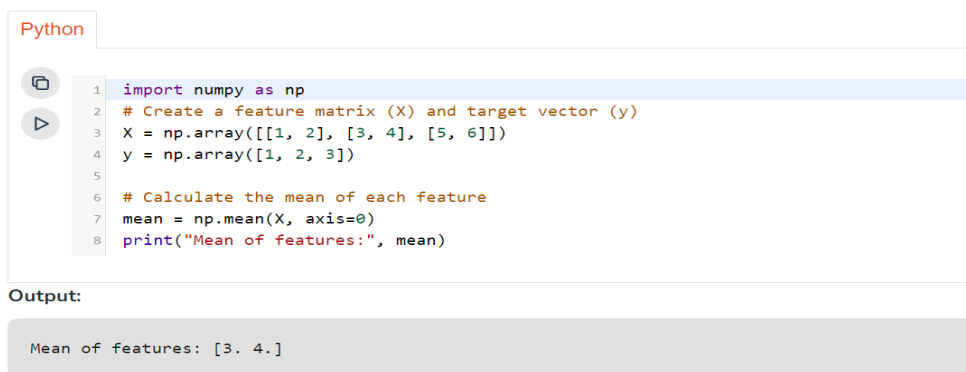
Drawbacks:

- Limited support for deep learning models.
- Not suitable for very large datasets due to lack of GPU support.
- May not perform as efficiently on high-dimensional data.

3) NumPy

NumPy is a very popular python library for large multi-dimensional array and matrix processing, with the help of a large collection of high-level mathematical functions. It is very useful for fundamental scientific computations in Machine Learning. It is particularly useful for linear algebra, Fourier transform, and random number capabilities. High-end libraries like TensorFlow uses NumPy internally for manipulation of Tensors.

Example: Linear Algebra Operations



```
Python
1 import numpy as np
2 # Create a feature matrix (X) and target vector (y)
3 X = np.array([[1, 2], [3, 4], [5, 6]])
4 y = np.array([1, 2, 3])
5
6 # Calculate the mean of each feature
7 mean = np.mean(X, axis=0)
8 print("Mean of features:", mean)

Output:
Mean of features: [3. 4.]
```

Figure 18. NumPy

4) Pandas

Pandas is a popular Python library for data analysis. It is not directly related to Machine Learning. As we know that the dataset must be prepared before training.

- In this case, Pandas comes handy as it was developed specifically for data extraction and preparation.
- It provides high-level data structures and wide variety tools for data analysis. It provides many inbuilt methods for grouping, combining and filtering data.

Example: Data Cleaning and Preparation

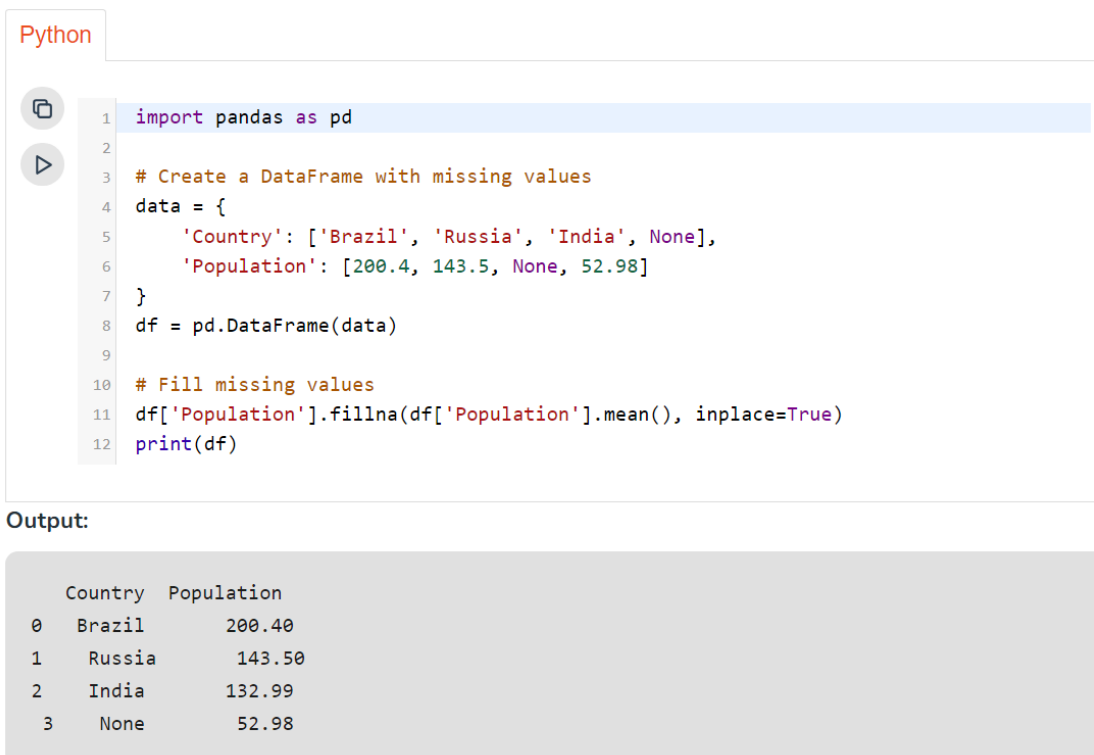


Figure 19. Pandas

5) Matplotlib

Matplotlib is a very popular Python library for data visualization. Like Pandas, it is not directly related to Machine Learning. It particularly comes in handy when a programmer wants to visualize the patterns in the data. It is a 2D plotting library used for creating 2D graphs and plots.

- A module named pyplot makes it easy for programmers for plotting as it provides features to control line styles, font properties, formatting axes, etc.

- It provides various kinds of graphs and plots for data visualization, viz., histogram, error charts, bar charts, etc,

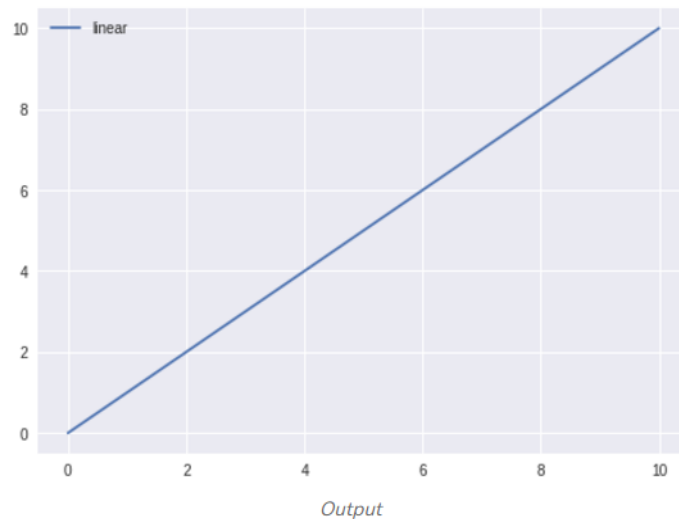
Example: Creating a linear Plot

Python

```
1 # Python program using Matplotlib
2 # for forming a linear plot
3
4 # importing the necessary packages and modules
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 # Prepare the data
9 x = np.linspace(0, 10, 100)
10
11 # Plot the data
12 plt.plot(x, x, label = 'linear')
13
14 # Add a legend
15 plt.legend()
16
17 # Show the plot
18 plt.show()
```

Figure 20. Matplotlib

Output:



5.4 Challenges to ML

1. Poor Quality of Data

Data plays a significant role in the machine learning process. One of the significant issues that machine learning professionals face is the absence of good quality data.

Unclean and noisy data can make the whole process extremely exhausting. We don't want our algorithm to make inaccurate or faulty predictions.

2. Underfitting of Training Data

This process occurs when data is unable to establish an accurate relationship between input and output variables. It simply means trying to fit in undersized jeans. It signifies the data is too simple to establish a precise relationship. To overcome this issue:

- Maximize the training time
- Enhance the complexity of the model
- Add more features to the data
- Reduce regular parameters
- Increasing the training time of model

3. Overfitting of Training Data

Overfitting refers to a machine learning model trained with a massive amount of data that negatively affect its performance. It is like trying to fit in Oversized jeans. Unfortunately, this is one of the significant issues faced by machine learning professionals. This means that the algorithm is trained with noisy and biased data, which will affect its overall performance. Let's understand this with the help of an example. Let's consider a model trained to differentiate between a cat, a rabbit, a dog, and a tiger. The training data contains 1000 cats, 1000 dogs, 1000 tigers, and 4000 Rabbits.

4. Machine Learning is a Complex Process

The machine learning industry is young and is continuously changing. Rapid hit and trial experiments are being carried on. The process is transforming, and hence there are high chances of error which makes the learning complex. It includes analysing the data, removing data bias, training data, applying complex mathematical calculations, and a lot more. Hence it is a really complicated process which is another big challenge for Machine learning professionals.

5. Lack of Training Data

The most important task you need to do in the machine learning process is to train the data to achieve an accurate output. Less amount training data will produce inaccurate or too biased predictions.

6. Slow Implementation

This is one of the common issues faced by machine learning professionals. The machine learning models are highly efficient in providing accurate results, but it takes a tremendous amount of time. Slow programs, data overload, and excessive requirements usually take a lot of time to provide accurate results. Further, it requires constant monitoring and maintenance to deliver the best output.

7. Imperfections in the Algorithm When Data Grows

So, you have found quality data, trained it amazingly, and the predictions are really concise and accurate. Yay, you have learned how to create a machine learning algorithm!! But wait, there is a twist; the model may become useless in the future as data grows. The best model of the present may become inaccurate in the coming Future and require further rearrangement. So you need regular monitoring and maintenance to keep the algorithm working. This is one of the most exhausting issues faced by machine learning professionals.

6. FUTURE TRENDS

6.1 Scope of ML

Machine learning (ML) is a branch of artificial intelligence that helps computers learn from data and make predictions or decisions. It has a wide range of applications across many industries. Let's take a look at some of the key areas where machine learning is making a big impact.

Predictive Analytics

Machine learning can analyze past data to predict future trends and outcomes. This is useful for anticipating customer behaviour, stock market changes, and equipment maintenance needs.

Natural Language Processing (NLP)

NLP helps computers understand and interact with human language. It powers chatbots, language translation, sentiment analysis, and speech recognition.

Computer Vision

Computer vision enables computers to understand visual information like images and videos. This technology is used in facial recognition, image classification, and object detection.

Recommendation Systems

Machine learning drives personalized recommendations on platforms like Netflix and Amazon. It suggests content and products based on user preferences and behaviour.

Fraud Detection

ML can analyze transaction data to identify patterns that may indicate fraud. This helps protect against financial fraud and other types of scams.

Healthcare

Machine learning aids in medical image analysis, disease diagnosis, and drug discovery. It also helps personalize treatment plans and improve healthcare operations.

Autonomous Vehicles

Self-driving cars use machine learning to interpret their surroundings and make real-time decisions. This technology is key to the future of transportation.

Finance

Machine learning is transforming finance with algorithmic trading, credit scoring, and risk assessment. Financial institutions use these tools to enhance their operations and services.

Cybersecurity

Machine learning helps detect malware and identify security breaches. It plays a crucial role in safeguarding data and systems from cyber threats.

Robotics

In robotics, machine learning enables robots to perform tasks like object manipulation and navigation. These capabilities are essential for advancing technology in industries like manufacturing and healthcare.

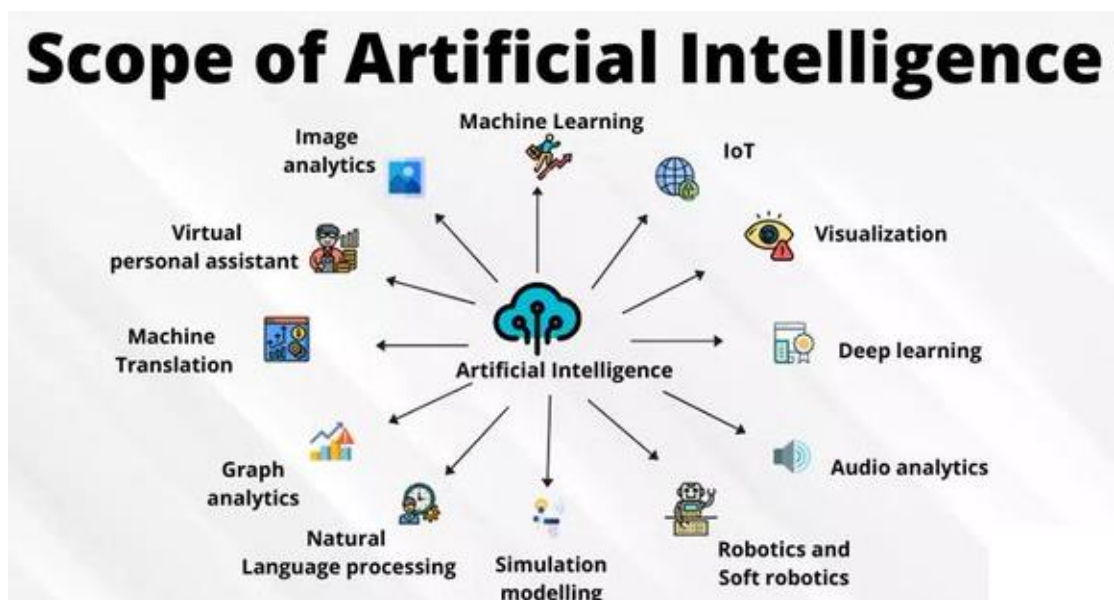


Figure 21. Scopes of ML

6.2 Benefits of ML

1. Natural language processing

Natural language processing (NLP) allows machine learning algorithms to process language-based inputs from humans, such as text-based messaging through an organisation's website.

2. Recognising images

Machine learning algorithms can learn to recognise images and then classify them into different categories. This means that they can recognise certain objects in an image and even recognise a face.

3. Data mining

Data mining refers to assessing data and finding patterns in it. This usually involves very large datasets containing raw data, which means data that hasn't undergone processing

4. Autonomous vehicles

Machine learning can allow an autonomous vehicle to learn how to safely navigate in the real world. It allows them to identify real-world objects accurately and react to them accordingly, meaning they can avoid collisions or disruptions for other vehicles or pedestrians.

5. Better advertising and marketing

Machine learning algorithms can predict which consumers are the most likely to actually buy a product. This is the process of customer segmentation, and having reliable information on buyer behaviour can make marketing and advertising campaigns much more effective.

6. Better products

Companies depend on feedback from consumers and reviewers to assess their products. Sales figures can indicate how popular the product is, but other variables like competitor products and marketing can also impact sales.

7. Speech recognition

Speech recognition is similar to natural language processing but focuses solely on verbal communication from humans. Machine learning can help speech recognition applications to better interpret voice-based inputs from consumers and others

8. Detecting fraud

Fraud detection is an important task for many organisations, especially businesses like banks that issue credit cards. Machine learning algorithms can analyse behaviour and spending patterns to identify potential instances of fraud, such as credit card theft and insurance fraud.

9. More accurate predictions

Making accurate predictions and forecasts is a key concern for many businesses and policymakers. These can be predictions about the stock market, the economy and consumer preferences.

10. Medical diagnoses

In the healthcare industry, machine learning can be useful for identifying patients who are at risk of certain conditions. Based on anonymous patient data from healthcare system records, machine learning algorithms can analyse patterns and combinations of lifestyle factors, histories and symptoms to assess how likely someone is to be at risk of a particular condition.

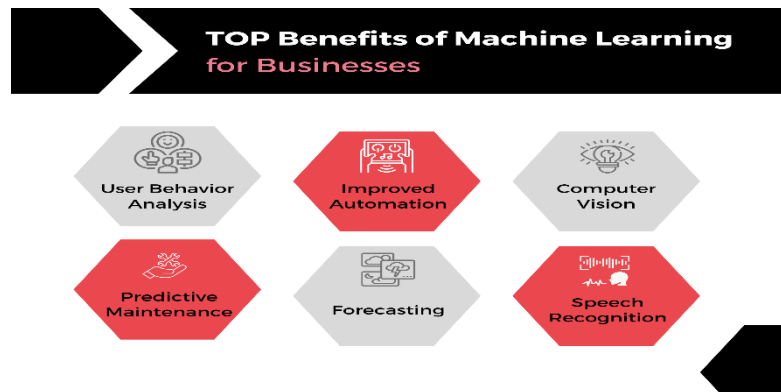


Figure 22. Benefits of ML

CONCLUSION

Python Programming:

Understanding the syntax of Python is great and all, and Python by itself is indeed a great language, but the fundamentals of Python aren't why Python is a successful language. There are plenty of fun-to-write languages that are just like Python, such as Ruby, Julia, even R. What makes Python the victor is the community and 3rd-party packages. While we can do a lot with just an installation of Python and the standard library, we can do infinitely more with all the 3rd party libraries out here.

Machine Learning:

In conclusion, machine learning is a powerful technology that allows computers to learn without explicit programming. By exploring different learning tasks and their applications, we gain a deeper understanding of how machine learning is shaping our world. From filtering your inbox to diagnosing diseases, machine learning is making a significant impact on various aspects of our lives.

REFERENCES

<https://iabac.org/>-(ML scopes)

<https://www.geeksforgeeks.org/> - (For Topics and pictures)

<https://www.simplilearn.com/> - (For content)

<https://chatgpt.com/>-(Codes)

<https://javatpoint.com/types-of-machine-learning>-(Algorithms)